TRANSPORT AND TELECOMMUNICATION INSTITUTE

Programming Languages Concepts

LAB #4

Topic: Procedures and Functions in PL

Student's name: Daniils
Student's surname: Buts
St. code: 78242

Lecturer: Karina Kostjkina

RIGA

TABLE OF CONTENTS

1. PURPOSE & TASK 3

2. PROGRAM CODE & RESULTS 4

2.1. Task 1 4

2.2. Task 2 5

2.3. Task 3 6

2.4. Task 4 7

CONCLUSIONS 8

1. PURPOSE & TASK

The purpose of this laboratory work is to take a look at the definition and usage of custom functions (procedures) in a programming language. Kotlin is our language of choice.

Tasks:

1. Write procedure Maximum for three real numbers that accepts from the keyboard.
2. Write two functions of a number raised to the square. In the first function parameter transmits by value, in the second – by reference.
3. Write a recursive function to calculate Fibonacci numbers. Apply a function to display the values of the Fibonacci sequence from 0 to 10.
4. Use recursive method to calculate the factorial of a number entered by the user.

2. PROGRAM CODE & RESULTS

2.1. Task 1

Write procedure Maximum for three real numbers that accepts from the keyboard.

Program Code:

```
import java.util.*

fun max(a: Double, b: Double) =
    if (a > b) a else b
….fun main() {
    val scanner = Scanner(System.`in`)

    print("Enter number A: ")
    val a = scanner.nextDouble()

    print("Enter number B: ")
    val b = scanner.nextDouble()

    print("Enter number C: ")
    val c = scanner.nextDouble()

    println("The largest number is ${….}")
}
```

Result:

```
Enter number A: 2.1
Enter number B: 6.7
Enter number C: -8.1
The largest number is 6.7

Process finished with exit code 0
```

Fig. 1 Calculation of the largest number

2.2. Task 2

Write two functions of a number raised to the square. In the first function parameter transmits by value, in the second – by reference.

Program Code:

```
data class Number(var value: Int)

// By value
fun sqr(n: Int): Int = n * n

// By reference
fun sqr(n: Number): Number = n.apply { value *= value }

fun main() {
    println("By value:")

    val n1 = 3
    ……    println("Object: $n2")
}
```

Result:

```
By value:
Result: 9
Object: 3
By reference:
Result: Number(value=9)
Object: Number(value=9)

Process finished with exit code 0
```

Fig. 2 Modifying data by value and by reference

2.3. Task 3

Write a recursive function to calculate Fibonacci numbers. Apply a function to display the values of the Fibonacci sequence from 0 to 10.

Program Code:

```
fun fib(n: UInt): UInt =
    when (n) {
        0u   -> 0u
        1u   -> 1u

    }

fun main() {
    for (i in 0u..10u) {
        print("${}
```

Result:

```
0 1 1 2 3 5 8 13 21 34 55
Process finished with exit code 0
```

Fig. 3 Numbers of the Fibonacci sequence

2.4. Task 4

Use recursive method to calculate the factorial of a number entered by the user.

Program Code:

```
import java.util.*

fun factorial(n: UInt): UInt =
    if (n == 0u)…..fun main() {
    val scanner = Scanner(System.`in`)

    print("Enter a positive integer: ")
    val n = scanner.nextInt().toUInt()
….}
```

Result:

```
Enter a positive integer: 5
5! = 120

Process finished with exit code 0
```

Fig. 4 Factorial of user entered number

CONCLUSIONS

The expected results were achieved. All program code, written in Kotlin, is included in this report, along with screenshots of the results.

We have shown several examples of defining custom functions in Kotlin. We have seen that function bodies may be implemented as expressions (the so-called *single-expression functions*) and are denoted by the "=" character. In fact, all functions in the above tasks were implemented as expressions, although we could have also used the more traditional C-style functions with explicit "return" statements and curly braces. However, our approach has the advantage of introducing less noise in the code, as well as being more declarative.

Another notable feature of the language is that the return types of single-expression functions may be inferred by the compiler, and thus omitted. We are making use of this feature in some examples to make the code shorter, although this has the disadvantage of being less explicit and thus allowing for more logical errors to occur. It is important to note that the compiler cannot infer types for recursive functions, therefore our implementations of "fib" and "factorial" must specify return types explicitly.

Lastly, all functions in Kotlin accept parameters by copy, but for non-primitive types only the reference is copied, therefore all parameters may be seen as "passed-by-reference". This makes it virtually impossible to distinguish between passing by value and by reference in Kotlin in the same way as in C or C++. Nonetheless, we demonstrated that it is possible to globally mutate the value of a parameter through the usage of custom classes.

On the one hand, Kotlin allows us to write cleaner and shorter code compared to the equivalent code in most other mainstream languages. On the other hand, the programmer must be careful not to make their code short to the point where its clarity is harmed.